



Agentic SDLC

From Copilot suggestions to autonomous development pipelines





“

The Agentic SDLC isn't about replacing developers.

It's about turning the development lifecycle into an **automated, quality-gated pipeline** where AI handles the routine and humans focus on **architecture, design, and approval.**



Feedback from the Hackathon!



The Progression



Labs

#	Lab	SDLC Phase	Autonomy	Folder
01	Code Generation Fundamentals	Code	● Human writes, Copilot suggests	lab01/
02	Testing with Copilot	Test	● Human directs, Copilot generates	lab02/
03	Debugging & Troubleshooting	Debug	● Human points, Copilot fixes	lab03/
04	Context Engineering	Cross-cutting	● Human configures, Copilot follows	lab04/
05	Code Review & Refactoring	Review	● Human asks, Agent researches	lab05/
06	Agent Mode	Code+Test+Debug	● Human approves, Agent implements	lab06/
07	Security in the SDLC	Security Gate	● Agent scans and enforces	lab07/
08	The Agentic SDLC Loop	Full Loop	●● Agent runs the full loop	lab08/

Increasingly
more autonomy
to the AI



Lab 01

Code Generation Fundamentals





Objective

Learning Objective

Master the fundamentals of GitHub Copilot as a **code generation tool**: inline completions, Chat interactions, and slash commands. By the end, you'll be fluent in the core features that every subsequent lab builds on.

What You'll Practice

- Inline completions (ghost text)
- Chat for code generation
- Slash commands
- Prompt crafting techniques
- GitHub Copilot CLI



Lab 02

Testing with Github Copilot





3 ways copilot generates tests (for *now*)

Inline Completions — Test as You Type

Start writing a test function name — Copilot completes the entire test body

The pattern

```
# Test that discount codes
# expire after 30 days
def test_apply_discount_

# Ghost text appears:
def test_apply_discount_expired_code():
    with pytest.raises(ValueError, match="expired"):
        apply_discount(
            price=100,
            code="SUMMER10",
            expired=True
        )
```

← Gray text = Copilot ghost text • Tab to accept

Tips for better completions



Descriptive function names

test_apply_discount_expired_code → Copilot knows what to generate just from the name



Add a comment first

Test that negative prices raise ValueError → acts as a mini spec for the completion



Use parametrize

Type @pytest.mark.parametrize and let Copilot fill in the test cases table



Cycle through suggestions

Alt+] / Alt+[to see alternative completions — useful when the first isn't quite right

The /tests Slash Command

Select a function → open inline chat → type /tests — Copilot generates the full test suite

How to use it

- 1 Open the function you want to test in VS Code
- 2 Select the entire function body (or just the signature)
- 3 Press Ctrl+I to open Inline Chat
- 4 Type /tests and hit Enter
- 5 Review, refine, and accept the generated tests

What Copilot generates

```
def test_calculate_total_basic():
    items = [{"price":10,"qty":2}]
    assert calculate_total(items) == 20

def test_calculate_total_with_discount():
    items = [{"price":100,"qty":1}]
    result = calculate_total(items, discount=0.1)
    assert result == pytest.approx(90.0)

def test_calculate_total_empty_items():
    with pytest.raises(ValueError):
        calculate_total([])
```

💡 Pro tip: Add context in your prompt — "/tests for edge cases including zero quantities and negative prices" gets much better results.

Copilot Chat — Directed Test Generation

Use Chat to describe exactly what you want — scenarios, edge cases, mocks

Basic generation

💬 @workspace /tests Generate unit tests for the `validate_order` function in `order_processor.py`. Include happy path and validation failures.

→ Generates ~6 tests covering required fields, type validation, and boundary values

Edge cases

💬 Add edge case tests for `validate_order`: empty cart, negative quantities, invalid email format, missing customer ID, and items with zero price.

→ Generates 5 targeted edge case tests with specific assert messages

With mocking

💬 Write tests for `process_payment` using `unittest.mock` to mock the payment gateway. Test success, declined card, and network timeout scenarios.

→ Generates mock setup, patch decorators, and 3 gateway response scenarios



Part 4 to come soon...
We will learn how to automate several
parts of the SDLC, including **testing**.



What Copilot Does Well vs. What to Watch For

✓ Copilot does well

- ✓ Happy path test generation
- ✓ Standard exception tests (ValueError, TypeError)
- ✓ `pytest.approx` for floating point comparisons
- ✓ Basic mock setup with `unittest.mock`
- ✓ Parametrize tables for data-driven tests
- ✓ Fixture boilerplate and setup/teardown

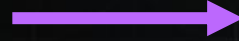
⚠ Watch out for

- × Business logic correctness — verify expected values
- × Domain-specific edge cases it doesn't know about
- × Complex multi-step integration scenarios
- × Performance / timing-sensitive assertions
- × Tests that pass but don't actually assert anything
- × Flaky tests with `time.sleep()` or random data

For future reference... you can save these bullet points for yourself:

Watch out for

- × Business logic correctness — verify expected values
- × Domain-specific edge cases it doesn't know about
- × Complex multi-step integration scenarios
- × Performance / timing-sensitive assertions
- × Tests that pass but don't actually assert anything
- × Flaky tests with `time.sleep()` or random data



This might be exactly
where your custom
agents and associated
skills will add the **MOST**
value



Today's Lab





Learning Objectives for Lab 2

Mock external dependencies (HTTP APIs, databases) using `unittest.mock`

Use fixtures, factories, and parametrize for clean, data-driven tests

Test async code with `AsyncMock` and `pytest-asyncio`

Find and fill coverage gaps with Copilot Agent Mode

Set CI coverage gates and explore Agentic Workflows

Use Copilot CLI (explain, suggest, task) for terminal-first testing

Run E2E tests with MCP servers (Playwright) from Agent Mode

Repo guidance

The screenshot displays the GitHub repository page for **ghcp-labs** by user **martaldsantos**. The repository is public and currently on the **main** branch, which has 1 branch and 0 tags. The commit history shows 11 commits, with the latest commit **f4b53e7** from yesterday. The file list includes **.github**, **lab01**, **lab02**, **lab03**, **lab04**, **lab05**, **lab06**, **lab07**, **lab08**, and **README.md**. The **README** file is selected, showing a banner for **GitHub Copilot Labs** with the text "Get your first steps on an Agentic SDLC" and the Microsoft logo. Below the banner, the title **GitHub Copilot Advanced Labs — The Agentic SDLC** is displayed, followed by a description: "A hands-on workshop that progressively builds Agentic SDLC skills. Across 8 labs (~1 hour each), participants go from using Copilot as a suggestion tool to orchestrating a fully autonomous development pipeline." The right sidebar contains sections for **About** (no description), **Releases** (no releases published), **Packages** (no packages published), **Contributors** (1 contributor: **martaldsantos**), and **Languages** (Python 100.0%).

ghcp-labs Public

main 1 Branch 0 Tags

Go to file Add file Code

martaldsantos Remove Lab 02 Testing with Copilot presentation file f4b53e7 · yesterday 11 Commits

.github	Add redlining validator and thumbnail generator scripts; enha...	yesterday
lab01	Add redlining validator and thumbnail generator scripts; enha...	yesterday
lab02	Remove Lab 02 Testing with Copilot presentation file	yesterday
lab03	Add lab reviewer and fixer agents; update lab instructions an...	3 days ago
lab04	Add lab reviewer and fixer agents; update lab instructions an...	3 days ago
lab05	Add lab reviewer and fixer agents; update lab instructions an...	3 days ago
lab06	Add lab reviewer and fixer agents; update lab instructions an...	3 days ago
lab07	Add lab reviewer and fixer agents; update lab instructions an...	3 days ago
lab08	Add lab reviewer and fixer agents; update lab instructions an...	3 days ago
README.md	Add lab reviewer and fixer agents; update lab instructions an...	3 days ago

README

GitHub Copilot Labs
Get your first steps on an Agentic SDLC

Microsoft

GitHub Copilot Advanced Labs — The Agentic SDLC

A hands-on workshop that progressively builds Agentic SDLC skills. Across 8 labs (~1 hour each), participants go from using Copilot as a suggestion tool to orchestrating a fully autonomous development pipeline.

About
No description, website, or topics provided.

Readme
Activity
0 stars
0 watching
0 forks

Releases
No releases published
[Create a new release](#)

Packages
No packages published
[Publish your first package](#)

Contributors 1
martaldsantos Marta dos Santos

Languages
Python 100.0%

<https://github.com/martaldsantos/ghcp-labs>



Thank you